

for loops

A **for loop** allows us to iterate over an object (such as a vector) and we can then perform and execute blocks of codes *for* every loop we go through. The syntax for a for loop is:

```
for (temporary_variable in object){  
  # Execute some code at every loop  
}
```

Let's start off by showing how to use a for loop with a vector:

For loop over a vector

We can think of looping through a vector in two different ways, the first way would be to create a temporary variable with the use of the **in** keyword:

In [1]:

```
vec <- c(1,2,3,4,5)
```

In [2]:

```
for (temp_var in vec){  
  print(temp_var)  
}
```

```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5
```

The other way would be to loop a numbered amount of times and then use indexing to continually grab from the vector:

In [3]:

```
for (i in 1:length(vec)){  
  print(vec[i])  
}
```

```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5
```

For loop over a list

We can do the same thing with a list:

In [4]:

```
li <- list(1,2,3,4,5)
```

In [8]:

```
for (temp_var in li){
```

```
for (temp_var in 1:5) {  
  print(temp_var)  
}
```

```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5
```

In [10]:

```
for (i in 1:length(li)){  
  print(li[[i]]) # Remember to use double brackets!  
}
```

```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5
```

For loop with a matrix

We can similarly loop through each individual element in a matrix:

In [14]:

```
mat <- matrix(1:25,nrow=5)  
mat
```

Out[14]:

1	6	11	16	21
2	7	12	17	22
3	8	13	18	23
4	9	14	19	24
5	10	15	20	25

In [15]:

```
for (num in mat) {  
  print(num)  
}
```

```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5  
[1] 6  
[1] 7  
[1] 8  
[1] 9  
[1] 10  
[1] 11  
[1] 12  
[1] 13  
[1] 14  
[1] 15  
[1] 16  
[1] 17  
[1] 18  
[1] 19  
[1] 20  
[1] 21  
[1] 22  
[1] 23  
[1] 24  
[1] 25
```

```
[1] 19
[1] 20
[1] 21
[1] 22
[1] 23
[1] 24
[1] 25
```

Nested for loops

We can nest for loops inside one another, however be careful when doing this, as every additional for loop nested inside another may cause a significant amount of additional time for your code to finish executing. For example:

In [28]:

```
for (row in 1:nrow(mat)) {
  for (col in 1:ncol(mat)) {
    print(paste('The element at row:', row, 'and col:', col, 'is', mat[row,col]))
  }
}
```

```
[1] "The element at row: 1 and col: 1 is 1"
[1] "The element at row: 1 and col: 2 is 6"
[1] "The element at row: 1 and col: 3 is 11"
[1] "The element at row: 1 and col: 4 is 16"
[1] "The element at row: 1 and col: 5 is 21"
[1] "The element at row: 2 and col: 1 is 2"
[1] "The element at row: 2 and col: 2 is 7"
[1] "The element at row: 2 and col: 3 is 12"
[1] "The element at row: 2 and col: 4 is 17"
[1] "The element at row: 2 and col: 5 is 22"
[1] "The element at row: 3 and col: 1 is 3"
[1] "The element at row: 3 and col: 2 is 8"
[1] "The element at row: 3 and col: 3 is 13"
[1] "The element at row: 3 and col: 4 is 18"
[1] "The element at row: 3 and col: 5 is 23"
[1] "The element at row: 4 and col: 1 is 4"
[1] "The element at row: 4 and col: 2 is 9"
[1] "The element at row: 4 and col: 3 is 14"
[1] "The element at row: 4 and col: 4 is 19"
[1] "The element at row: 4 and col: 5 is 24"
[1] "The element at row: 5 and col: 1 is 5"
[1] "The element at row: 5 and col: 2 is 10"
[1] "The element at row: 5 and col: 3 is 15"
[1] "The element at row: 5 and col: 4 is 20"
[1] "The element at row: 5 and col: 5 is 25"
```

Great! That's it for for loops, we'll test you on this knowledge later on!